

Algorytmiczna Analiza Danych

Ćwiczenia 15

30 stycznia 2025 r.

Adrian Herda
Politechnika Wrocławska

1. Ćwiczenie 54 – Algorytm K-Means

1.1. Działanie algorytmu

Algorytm k-means służy do podziału zbioru na k rozłącznych klastrow. Jego działanie można opisać w następujących krokach:

1. **Inicjalizacja** - Wybierz losowo k punktów ze zbioru danych. Położą one za początkowe centroidy.
2. **Przypisanie** - Dla każdego punktu w zbiorze danych oblicz jego odległości do wszystkich k centroidów. Przypisz punkt do klastra, reprezentowanego przez centroid, do którego ma najmniejszą odległość.
3. **Aktualizacja** - Oblicz współrzędne centroidów jako średnią arytmetyczną wszystkich punktów przypisanych do danego klastra.
4. **Powtórzenie** - Powtarzaj kroki 2 i 3, aż do spełnienia warunku stopu.

1.2. Złożoność obliczeniowa

- **Czasowa:** $O(t \cdot k \cdot n \cdot d)$, gdzie:
 - t : liczba iteracji,
 - k : liczba klastrow,
 - n : liczba punktów danych,
 - d : wymiarowość danych.

Zazwyczaj $k, t, d \ll n$, więc algorytm jest liniowy względem liczby punktów.

- **Pamięciowa:** $O((n + k) \cdot d)$ ponieważ musimy przechowywać d parametrów, jeden na każdy wymiar, dla każdego punktu i centroidu.

1.3. Dlaczego Potrzebne jest wielokrotne uruchomienie

Algorytm jest heurystyką, to oznacza że nie gwarantuje znalezienia globalnego minimum. Ponad to algorytm bazuje na wyborze losowym początkowych centroidów a jako że wynik końcowy jest w dużej mierze zależny od początkowych wartości, to jest też zależny od tej losowości. Kilukrotne uruchomienie z różnymi losowymi inicjalizacjami zwiększa szansę na znalezienie optymalnego podziału.

1.4. Pechowy wybór centroidów

Przykład: Załóżmy że są dwa naturalne klastry punktów na wykresie, jeśli dwa początkowe centroidy znajdują się w klastrze A, to K-Means nie zmieni tego i podzieli A na dwa klastry a B przypisze do jednego z nich, co jest oczywiście błędne.

2. Ćwiczenie 55 – Dowód na postać centroidu

a) **Theorem 2.1.** $\sum_{i=1}^k \sum_{x \in C_i} (x - c_i)^2$, centroid ma postać średniej punktów w klastrze

Proof. Funkcja kosztu dla dowolnego klastra i to:

$$J(c_i) = \sum_{x \in C_i} (x - c_i)^2 \quad (2.1)$$

Aby znaleźć minimum, obliczamy pochodną po c i przyrównujemy do 0.

$$\begin{aligned} \frac{dJ}{dc_i} &= \sum_{x \in C_i} (x^2 - 2xc_i + c_i^2)' \\ &= \sum_{x \in C_i} (-2x + 2c_i) \\ &= 2 \cdot |C_i| \cdot c_i - 2 \cdot \sum_{x \in C_i} x = 0 \end{aligned} \quad (2.2)$$

$$\begin{aligned} 2 \cdot |C_i| \cdot c_i &= 2 \cdot \sum_{x \in C_i} x \\ c_i &= \frac{1}{|C_i|} \sum_{x \in C_i} x \end{aligned}$$

■

b) **Theorem 2.2.** $\sum_{i=1}^k \sum_{x \in C_i} |x - c_i|$, centroid ma postać mediany punktów w klastrze

Podobnie jak w podpunkcie a) obliczamy pochodną kosztu centroidu c_i :

$$J(c_i) = \sum_{x \in C_i} |x - c_i| \quad (2.3)$$

$$\begin{aligned} \frac{dJ}{dc} &= \sum_{x \in C} \frac{d}{dc} |x - c_i| \\ &= \sum_{x \in C} -\text{sgn}(x - c_i) = 0 \end{aligned} \quad (2.4)$$

Suma znaków wyrażeń $x - c_i$ będzie równa zero tylko i wyłącznie wtedy kiedy będzie tyle samo wyrażeń -1 czyli $x < c_i$ co wyrażeń $+1$ produkowanych przez $x > c_i$:

$$|\{x \in C_i : x < c_i\}| = |\{x \in C_i : x > c_i\}| \quad (2.5)$$

a to z kolei znaczy że c_i dzieli zbiór punktów C_i idealnie na poł co jest definicją mediany.

3. Ćwiczenie 56 – DBSCAN

3.1. Działanie DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) grupuje punkty na podstawie gęstości ich rozmieszczenia.

1. Dla każdego punktu sprawdzamy jego ε -sąsiedztwo.
2. Jeśli punkt ma co najmniej $MinPts$ sąsiadów, staje się **rdzeniem** (core point) i tworzy nowy klaster.
3. Do klastra dołączane są wszystkie punkty osiągalne gęstościowo (sąsiedzi rdzenia, sąsiedzi ich sąsiadów itd.).
4. Punkty, które nie są rdzeniami, ale leżą w sąsiedztwie rdzenia, to **punkty brzegowe**.
5. Punkty nieprzypisane do żadnego klastra to **szum** (noise).

3.2. Ustalanie parametrów

- **MinPts:** Zazwyczaj ustala się jako $2 \cdot d$ (gdzie d to wymiar danych). Dla większych zbiorów z szumem warto zwiększyć tę wartość.
- **ε (epsilon):** Stosuje się metodę **k-distance graph**. Obliczamy odległość każdego punktu do jego k -tego najbliższego sąsiada (gdzie $k = MinPts - 1$). Sortujemy te odległości i wykreślamy. Punkt „kolanka” (gwałtownego wzrostu wykresu) wskazuje optymalne ε .

3.3. Porównanie k-means i DBSCAN

k-means	DBSCAN
Wymaga podania liczby klastrów k .	Sam ustala liczbę klastrów.
Zakłada kulisty kształt klastrów.	Wykrywa klastry o dowolnych kształtach.
Wrażliwy na szum (przypisuje wszystko).	Odporny na szum (oznacza jako noise).
Szybki ($O(n)$).	Wolniejszy w pesymistycznym przypadku ($O(n^2)$), ale $O(n \log n)$ z indeksowaniem.
Działa dobrze, gdy gęstość jest zmienna.	Problemy przy klastrach o bardzo różnej gęstości.

3.4. Co lepsze dla dokumentów (dane wielowymiarowe)?

Dla zbioru dokumentów (bardzo wysoki wymiar, rzadkie dane) zazwyczaj **k-means** (często w wariacie sferycznym lub z metryką cosinusową) okazuje się skuteczniejszy. DBSCAN w bardzo wielu wymiarach cierpi na „klątwę wymiarowości” – odległości między wszystkimi punktami stają się do siebie zbliżone, co utrudnia dobranie sensownego ε i odróżnienie gęstych regionów od rzadkich.

4. Ćwiczenie 57 — Klasteryzacja hierarchiczna

4.1. Działanie

Klasteryzacja hierarchiczna (aglomeracyjna) zaczyna od sytuacji, gdzie każdy punkt jest osobnym klastrem. W każdym kroku łączy dwa najbliższe klastry w jeden, aż wszystkie punkty znajdą się w jednym klastrze. Wynikiem jest dendrogram (drzewo połączeń).

4.1.1. Single Linkage

Odległość między klastrami to odległość między **najbliższymi** punktami z tych klastrów.

1. Najmniejsze odległości to 2 (pary $\{1,3\}$ oraz $\{8,10\}$). Łączymy je.

- Klastry: $\{1,3\}$, $\{8,10\}$, $\{16\}$.

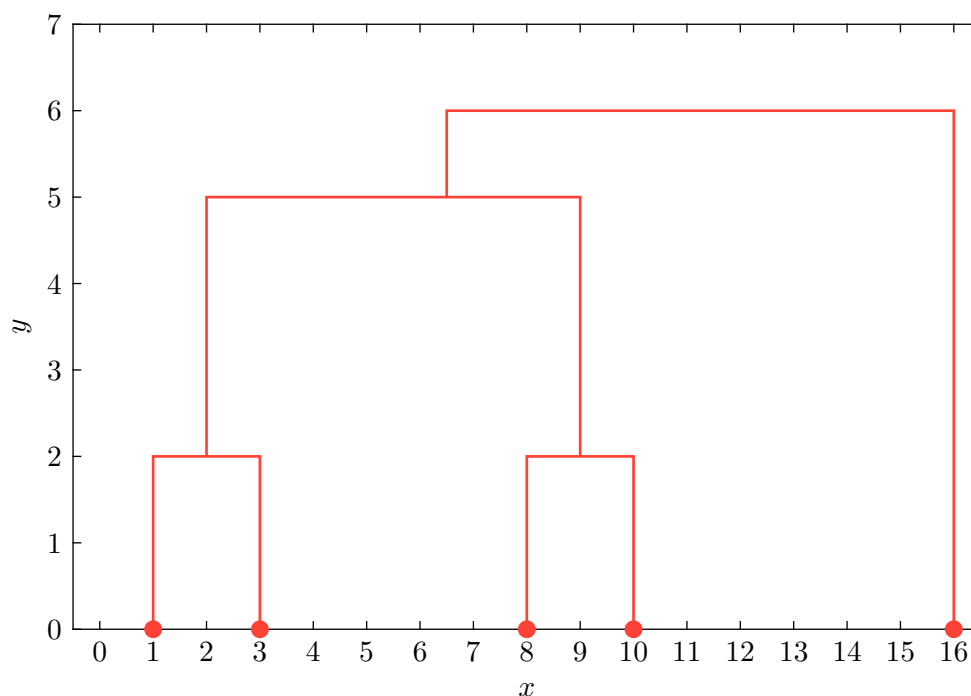
2. Odległości między nowymi klastrami:

- $d(\{1,3\}, \{8,10\}) = d(3,8) = 5$
- $d(\{8,10\}, \{16\}) = d(10,16) = 6$

Najmniejsza to 5. Łączymy $\{1,3\}$ i $\{8,10\}$.

- Klastry: $\{1,3,8,10\}$, $\{16\}$.

3. Łączymy wszystko z $\{16\}$. Odległość: $d(10,16) = 6$.



4.1.2. Complete Linkage

Odległość między klastrami to odległość między **najdalszymi** punktami.

1. Najmniejsze odległości to 2. Łączymy $\{1,3\}$ i $\{8,10\}$.

- Klastry: $\{1,3\}$, $\{8,10\}$, $\{16\}$.

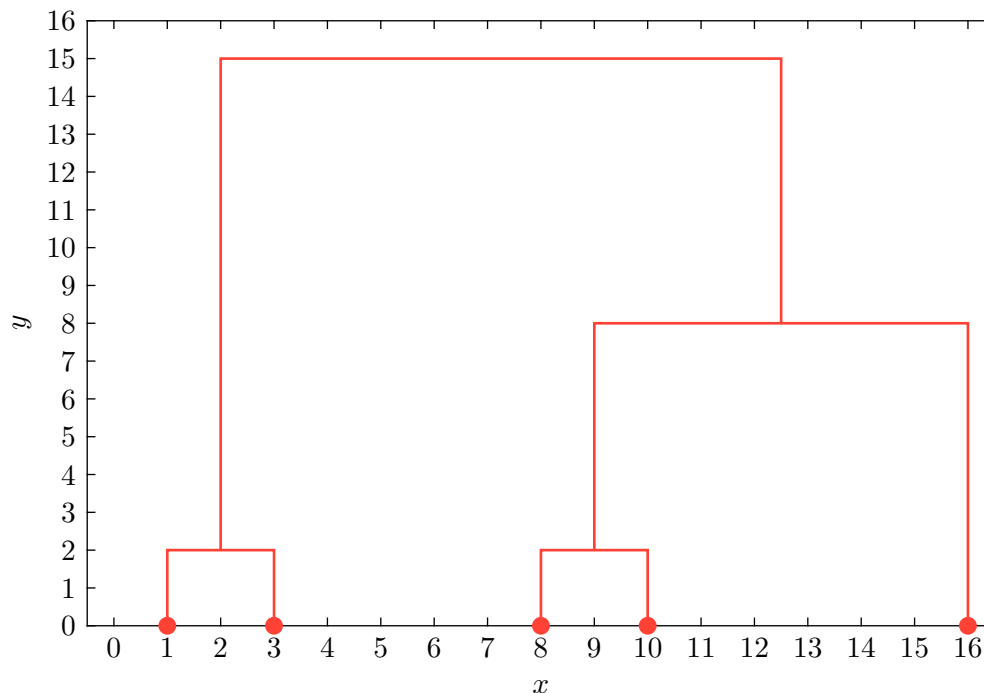
2. Aktualizujemy macierz odległości (bierzemy maksimum):

- $d(\{1,3\}, \{8,10\}) = d(1,10) = 9$
- $d(\{8,10\}, \{16\}) = d(8,16) = 8$
- $d(\{1,3\}, \{16\}) = d(1,16) = 15$

Najmniejsza z tych wartości to 8. Łączymy $\{8,10\}$ i $\{16\}$.

- Klasy: $\{1, 3\}, \{8, 10, 16\}$.

3. Łączymy pozostałe dwie grupy. Odległość to $\max(d(1, 16), d(3, 16)) = 15$.



4.2. Ustalanie liczby klastrów z dendrogramu

Należy „ciąć” dendrogram w miejscu, gdzie występuje najdłuższa pionowa linia (największy skok odległości) bez przecinania poziomych linii łączenia. Sugeruje to dużą separację między powstałymi grupami.

5. Ćwiczenie 58 – Wybór metody dla klastrów o nietypowych kształtach

5.1. Poprawne metody:

- **DBSCAN** z odpowiednio dobranymi parametrami: DBSCAN grupuje na podstawie gęstości. Ponieważ linia i okrąg są gęstymi zbiorami punktów oddzielonymi pustą przestrzenią, DBSCAN poradzi sobie bez problemów.
- **Klasteryzacja hierarchiczna - single linkage**: Efektem algorytmu single linkage jest łańcuchowanie się punktów. Jeśli jakiś punkt nie należy do klastra ale jego bliski sąsiad należy, to klaster przyjmie też ten punkt nie zwracając uwagi na to jak daleko jest od centroidu.

5.2. Niepoprawne metody:

- **K-Means**: dąży do tworzenia klastrów wypukłych, najlepiej w kształcie okręgu, o podobnych wielkościach. W tym przypadku linia nie jest okrągła więc K-Means najprawdopodobniej ją podzieli.
- **Klasteryzacja hierarchiczna - complete linkage**: Preferuje małe zwarte zbiory. Najprawdopodobniej będzie próbował podzielić linię na mniejsze zbiory.